Multimodelcode2

```
from google.colab import drive
drive.mount('/content/drive')

%cd /content/drive/MyDrive/DS301-Project
```

```
Mounted at /content/drive
/content/drive/MyDrive/DS301-Project
```

Code2 Notebook

This notebook extends `code1.ipynb` by adding a **multimodal fusion model** combining **text (BERT)** and **audio (MFCC)** features for emotion classification.

We will:

1. Load preprocessed text and audio features
2. Build a multimodal classifier
3. Train and evaluate on MELD dataset

```
import pickle
import json

# Load preprocessed features
with open('/content/drive/MyDrive/DS301-Project/meld_text_audio_features.pkl', 'rb') as f:
    data = pickle.load(f)

# Load label mapping
with open('/content/drive/MyDrive/DS301-Project/code/label2id.json', 'r') as f:
    label2id = json.load(f)
id2label = {v: k for k, v in label2id.items()}
```

```
from torch.utils.data import Dataset, DataLoader
import torch

class MELDMultiModalDataset(Dataset):
    def __init__(self, data):
        self.data = data

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample = self.data[idx]
        input_ids = sample['text_tokens']['input_ids'].squeeze(0)
        attention_mask = sample['text_tokens']['attention_mask'].squeeze(0)
        audio_mfcc = torch.tensor(sample['audio_mfcc'], dtype=torch.float32)
        label = label2id[sample['label']]
        return input_ids, attention_mask, audio_mfcc, torch.tensor(label)
```

```
from torch.nn.utils.rnn import pad_sequence

def collate_fn(batch):
    input_ids = [item[0] for item in batch]
    attention_masks = [item[1] for item in batch]
    audio_mfccs = [item[2] for item in batch]
    labels = [item[3] for item in batch]

    input_ids_padded = pad_sequence(input_ids, batch_first=True, padding_value=0)
    attention_masks_padded = pad_sequence(attention_masks, batch_first=True, padding_value=0)
    audio_mfccs_stacked = torch.stack(audio_mfccs)
    labels_tensor = torch.stack(labels)

    return input_ids_padded, attention_masks_padded, audio_mfccs_stacked, labels_tensor
```

```
dataset = MELDMultiModalDataset(data)
dataloader = DataLoader(dataset, batch_size=8, shuffle=True, collate_fn=collate_fn)
```

```
import torch.nn as nn

class MultiModalClassifier(nn.Module):
    def __init__(self, text_hidden_dim, audio_feat_dim, num_labels):
        super().__init__()
        self.text_fc = nn.Linear(text_hidden_dim, 128)
        self.audio_fc = nn.Linear(audio_feat_dim, 128)
        self.classifier = nn.Sequential(
            nn.ReLU(),
            nn.Linear(128*2, 64),
            nn.ReLU(),
            nn.Linear(64, num_labels)
        )

    def forward(self, text_emb, audio_feat):
        text_out = self.text_fc(text_emb)
        audio_out = self.audio_fc(audio_feat)
        fused = torch.cat((text_out, audio_out), dim=1)
        return self.classifier(fused)
```

```
from transformers import BertModel

bert = BertModel.from_pretrained('bert-base-uncased')
bert.eval()  # freeze BERT initially or allow fine-tuning later
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

```
config.json:   0%|          | 0.00/570 [00:00<?, ?B/s]
```

```
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP
download. For better performance, install the package with: `pip install huggingface_hub[hf_xet]` or `pip
install hf_xet`
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not
installed. Falling back to regular HTTP download. For better performance, install the package with: `pip
install huggingface_hub[hf_xet]` or `pip install hf_xet`
```

```
model.safetensors:   0%|          | 0.00/440M [00:00<?, ?B/s]
```

```
BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 768, padding_idx=0)
    (position_embeddings): Embedding(512, 768)
    (token_type_embeddings): Embedding(2, 768)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-11): 12 x BertLayer(
        (attention): BertAttention(
          (self): BertSdpaSelfAttention(
            (query): Linear(in_features=768, out_features=768, bias=True)
            (key): Linear(in_features=768, out_features=768, bias=True)
            (value): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=768, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=768, out_features=3072, bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=3072, out_features=768, bias=True)
          (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
```

```
  (pooler): BertPooler(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (activation): Tanh()
  )
)
```

Next steps:

- Pass input_ids & attention_masks through `bert` to get embeddings
- Feed embeddings + audio_mfcc to `MultiModalClassifier`
- Define optimizer, loss
- Training & evaluation loop

```
input_ids = torch.load('input_ids.pt')
attention_mask = torch.load('attention_mask.pt')

print(input_ids.shape)
```

```
torch.Size([100, 44])
```

```
import torch
import pickle

with open('/content/drive/MyDrive/DS301-Project/meld_text_audio_features.pkl', 'rb') as f:
    data = pickle.load(f)

audio_features = torch.stack([torch.tensor(d['audio_mfcc'], dtype=torch.float32).mean(dim=1) for d in data])

print(audio_features.shape)  # num_samples, mfcc_dim

# Save
torch.save(audio_features, '/content/drive/MyDrive/DS301-Project/code/audio_features.pt')
print("Saved audio_features.pt")
```

```
torch.Size([100, 13])
Saved audio_features.pt
```

```
with torch.no_grad():
    bert_outputs = bert(input_ids=input_ids, attention_mask=attention_mask)
    text_embeddings = bert_outputs.pooler_output

# Load audio features
audio_features = torch.load('code/audio_features.pt')

print(f"Text embeddings shape: {text_embeddings.shape}")
print(f"Audio features shape: {audio_features.shape}")

# Concatenate text and audio features
fused_features = torch.cat([text_embeddings, audio_features], dim=1)
print(f"Fused features shape: {fused_features.shape}")
```

```
Text embeddings shape: torch.Size([100, 768])
Audio features shape: torch.Size([100, 13])
Fused features shape: torch.Size([100, 781])
```

```
# define number of labels
num_labels = len(label2id)

# simple classifier
classifier = nn.Linear(768, num_labels)
logits = classifier(text_embeddings)

# prediction
preds = torch.argmax(logits, dim=1)
print(preds)
```

```
tensor([0, 0, 0, 0, 0, 2, 0, 2, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 3, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 2, 0,
        0, 0, 0, 2, 0, 0, 2, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0])
```

```
import json

# Load label mapping (needed for num_labels and decoding)
with open('/content/drive/MyDrive/DS301-Project/code/label2id.json', 'r') as f:
    label2id = json.load(f)

# Load data again to get labels
with open('/content/drive/MyDrive/DS301-Project/meld_text_audio_features.pkl', 'rb') as f:
    data = pickle.load(f)

# Extract labels into tensor
labels = torch.tensor([label2id[d['label']] for d in data])
```

incorporate audio_features.pt into model

```
audio_features = torch.load('/content/drive/MyDrive/DS301-Project/code/audio_features.pt')
print(audio_features.shape)  # num_samples, mfcc_dim
```

```
torch.Size([100, 13])
```

```
combined_features = torch.cat([text_embeddings, audio_features], dim=1)  # dim=1 = feature axis
print(combined_features.shape)
```

```
torch.Size([100, 781])
```

```
input_dim = combined_features.shape[1]
classifier = nn.Linear(input_dim, num_labels)
```

```
input_dim = combined_features.shape[1]
classifier = nn.Sequential(
    nn.Linear(input_dim, 256),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, num_labels)
)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(classifier.parameters(), lr=1e-4)

for epoch in range(20):
    classifier.train()
    optimizer.zero_grad()

    logits = classifier(combined_features)
    loss = criterion(logits, labels.clone().detach())
    preds = torch.argmax(logits, dim=1)
    correct = (preds == labels).sum().item()
    accuracy = correct / labels.size(0)

    loss.backward()
    optimizer.step()

    print(f"Epoch {epoch+1}: Loss = {loss.item():.4f}, Accuracy = {accuracy:.2%}")
```

```
Epoch 1: Loss = 1.9920, Accuracy = 19.00%
Epoch 2: Loss = 1.8690, Accuracy = 22.00%
Epoch 3: Loss = 1.8239, Accuracy = 34.00%
Epoch 4: Loss = 1.7019, Accuracy = 31.00%
Epoch 5: Loss = 1.7073, Accuracy = 31.00%
Epoch 6: Loss = 1.6298, Accuracy = 39.00%
Epoch 7: Loss = 1.7405, Accuracy = 39.00%
Epoch 8: Loss = 1.7225, Accuracy = 44.00%
Epoch 9: Loss = 1.5675, Accuracy = 50.00%
Epoch 10: Loss = 1.6316, Accuracy = 49.00%
Epoch 11: Loss = 1.6549, Accuracy = 49.00%
Epoch 12: Loss = 1.6426, Accuracy = 49.00%
Epoch 13: Loss = 1.6312, Accuracy = 49.00%
Epoch 14: Loss = 1.7825, Accuracy = 46.00%
Epoch 15: Loss = 1.6037, Accuracy = 50.00%
Epoch 16: Loss = 1.7242, Accuracy = 49.00%
Epoch 17: Loss = 1.5511, Accuracy = 50.00%
Epoch 18: Loss = 1.6078, Accuracy = 42.00%
Epoch 19: Loss = 1.6149, Accuracy = 48.00%
Epoch 20: Loss = 1.6569, Accuracy = 45.00%
```

```
logits = classifier(combined_features)
loss = criterion(logits, labels)
```

```
# Normalize audio features
audio_features_norm = (audio_features - audio_features.mean(dim=0)) / (audio_features.std(dim=0) + 1e-6)

# Concatenate normalized audio with text embeddings
combined_features = torch.cat([text_embeddings, audio_features_norm], dim=1)

# Define classifier with hidden layer + ReLU
input_dim = combined_features.shape[1]
classifier = nn.Sequential(
    nn.Linear(input_dim, 256),
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, num_labels)
)


# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(classifier.parameters(), lr=1e-4)

# Training loop
for epoch in range(20):
    classifier.train()
    optimizer.zero_grad()

    logits = classifier(combined_features)
    loss = criterion(logits, labels)

    preds = torch.argmax(logits, dim=1)
    correct = (preds == labels).sum().item()
    accuracy = correct / labels.size(0)

    loss.backward()
    optimizer.step()

    print(f"Epoch {epoch+1}: Loss = {loss.item():.4f}, Accuracy = {accuracy:.2%}")
```

```
Epoch 1: Loss = 1.9930, Accuracy = 9.00%
Epoch 2: Loss = 1.9618, Accuracy = 9.00%
Epoch 3: Loss = 1.9332, Accuracy = 10.00%
Epoch 4: Loss = 1.9067, Accuracy = 10.00%
Epoch 5: Loss = 1.8815, Accuracy = 37.00%
Epoch 6: Loss = 1.8575, Accuracy = 50.00%
Epoch 7: Loss = 1.8349, Accuracy = 52.00%
Epoch 8: Loss = 1.8137, Accuracy = 53.00%
Epoch 9: Loss = 1.7929, Accuracy = 54.00%
Epoch 10: Loss = 1.7723, Accuracy = 53.00%
Epoch 11: Loss = 1.7516, Accuracy = 53.00%
Epoch 12: Loss = 1.7310, Accuracy = 53.00%
Epoch 13: Loss = 1.7102, Accuracy = 53.00%
Epoch 14: Loss = 1.6892, Accuracy = 53.00%
Epoch 15: Loss = 1.6682, Accuracy = 53.00%
Epoch 16: Loss = 1.6475, Accuracy = 52.00%
Epoch 17: Loss = 1.6271, Accuracy = 52.00%
Epoch 18: Loss = 1.6072, Accuracy = 52.00%
Epoch 19: Loss = 1.5878, Accuracy = 52.00%
Epoch 20: Loss = 1.5692, Accuracy = 52.00%
```

To Further investigate, let's run an audio-only training and validation.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# Load saved MFCC features
with open('/content/drive/MyDrive/DS301-Project/meld_text_audio_features.pkl', 'rb') as f:
    data = pickle.load(f)

# Pad/truncate MFCCs to fixed length
max_len = max(d['audio_mfcc'].shape[1] for d in data)
padded_features = []
for d in data:
    mfcc = torch.tensor(d['audio_mfcc'], dtype=torch.float32)
    time_steps = mfcc.shape[1]
    if time_steps < max_len:
        pad_width = max_len - time_steps
        mfcc = F.pad(mfcc, (0, pad_width))
    else:
        mfcc = mfcc[:, :max_len]
    padded_features.append(mfcc)

audio_features = torch.stack(padded_features)
print(audio_features.shape)

# Flatten features for feeding into MLP
audio_features_flat = audio_features.view(audio_features.shape[0], -1)
input_dim = audio_features_flat.shape[1]

# Prepare labels
label2id = json.load(open('/content/drive/MyDrive/DS301-Project/code/label2id.json'))
labels = torch.tensor([label2id[d['label']] for d in data])

num_labels = len(label2id)

# Define classifier
classifier = nn.Sequential(
    nn.Linear(input_dim, 256),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, num_labels)
)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(classifier.parameters(), lr=1e-4)

# Training
for epoch in range(20):
    classifier.train()
    optimizer.zero_grad()

    logits = classifier(audio_features_flat)
    loss = criterion(logits, labels.clone().detach())

    preds = torch.argmax(logits, dim=1)
    correct = (preds == labels).sum().item()
    accuracy = correct / labels.size(0)

    loss.backward()
    optimizer.step()

    print(f"Epoch {epoch+1}: Loss = {loss.item():.4f}, Accuracy = {accuracy:.2%}")
```

```
torch.Size([100, 13, 401])
Epoch 1: Loss = 9.0137, Accuracy = 10.00%
Epoch 2: Loss = 5.7171, Accuracy = 11.00%
Epoch 3: Loss = 4.2682, Accuracy = 32.00%
Epoch 4: Loss = 3.5020, Accuracy = 46.00%
Epoch 5: Loss = 4.1964, Accuracy = 41.00%
Epoch 6: Loss = 3.5363, Accuracy = 42.00%
Epoch 7: Loss = 2.7567, Accuracy = 48.00%
Epoch 8: Loss = 2.7225, Accuracy = 45.00%
Epoch 9: Loss = 2.1840, Accuracy = 48.00%
Epoch 10: Loss = 2.0531, Accuracy = 42.00%
Epoch 11: Loss = 1.8790, Accuracy = 46.00%
Epoch 12: Loss = 1.7845, Accuracy = 42.00%
Epoch 13: Loss = 1.5702, Accuracy = 55.00%
Epoch 14: Loss = 1.6945, Accuracy = 56.00%
Epoch 15: Loss = 1.3058, Accuracy = 56.00%
Epoch 16: Loss = 1.2443, Accuracy = 57.00%
Epoch 17: Loss = 1.2325, Accuracy = 60.00%
Epoch 18: Loss = 1.0825, Accuracy = 64.00%
Epoch 19: Loss = 1.1311, Accuracy = 63.00%
Epoch 20: Loss = 1.1309, Accuracy = 63.00%
```

```
input_dim = audio_features.shape[1] * audio_features.shape[2]

audio_only_classifier = nn.Sequential(
    nn.Linear(input_dim, 256),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(128, num_labels)
)

audio_only_classifier.eval()

correct = 0
total = 0

with torch.no_grad():
    # Flatten the audio_features for input to linear layer
    audio_features_flat = audio_features.view(audio_features.shape[0], -1)

    logits = audio_only_classifier(audio_features_flat)
    preds = torch.argmax(logits, dim=1)

    correct = (preds == labels).sum().item()
    total = labels.size(0)

accuracy = correct / total
print(f"Audio-only model → Test Accuracy: {accuracy:.2%}")
```

```
🎵 Audio-only model → Test Accuracy: 13.00%
```